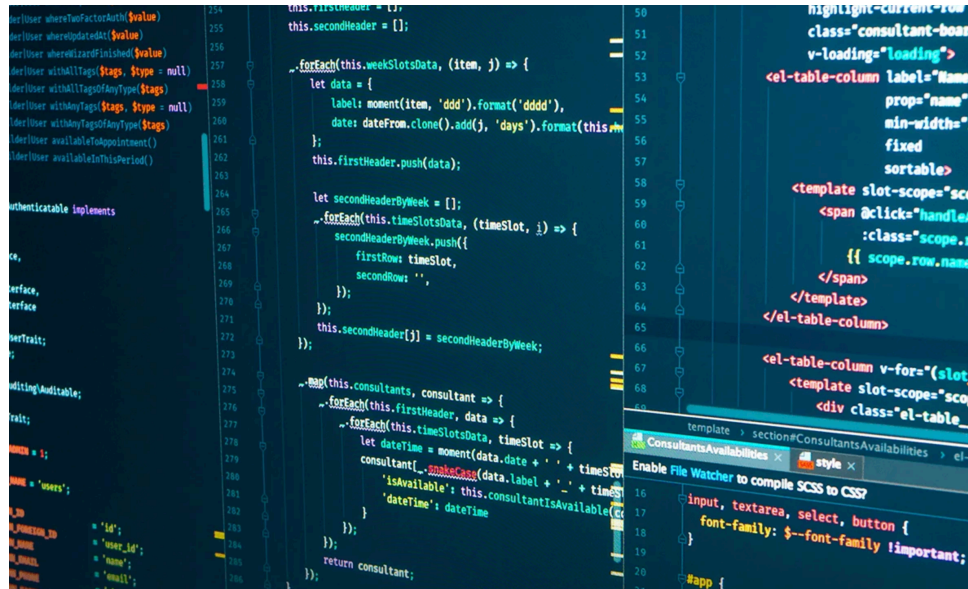


Llenguatges de Programació

Introducció a la compilació



Jordi Petit i Gerard Escudero

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Objectius

- Conèixer l'estructural general d'un compilador, les seves principals etapes, i la seva organització.
- Conèixer l'existència d'eines per ajudar a crear compiladors (usarem **ANTLR**).
- A la pràctica, ens limitem a crear petits processadors de llenguatges:
 1. Definició del vocabulari,
 2. Definició de la gramàtica,
 3. Generació de l'arbre de sintaxi abstracta,
 4. Interpretació a través del recorregut de l'arbre.
- El curs de Compiladors aprofundeix molts més els continguts.
- El curs de Teoria de la Computació n'ofereix els fonaments teòrics.

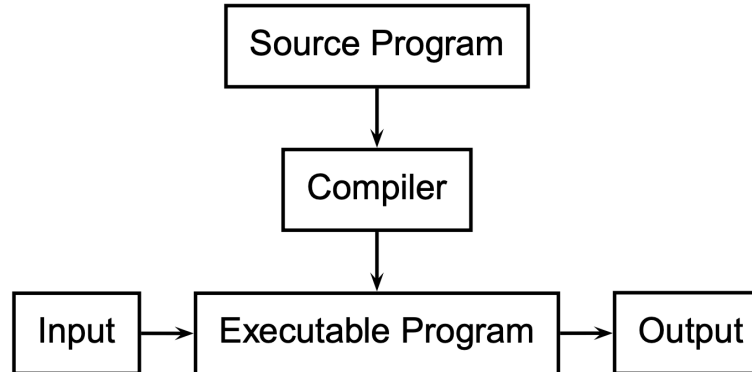
Crèdits

Gran part del material d'aquestes diapositives sobre compilació s'ha extret de les que va elaborar el professor **Stephen A. Edwards** (Universitat de Columbia) per l'assignatura COMS W4115 (Programming Languages and Translators) i que el professor **Jordi Cortadella** (UPC) va adaptar per l'assignatura de Compiladors. També s'ha extret material de les transparències del professor **Fernando Orejas** (UPC).

Contingut

- Introducció
- Visió general
 - Processadors de llenguatges
 - Sintaxi
 - Semàntica
 - Flux de compilació
 - Eines
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Compiladors



Un **compilador** és un programa que tradueix programes escrits en un LP d'alt nivell a codi màquina (o, en general, a codi de baix nivell).

Exemples: GCC, CLANG, go, ghc, ...

Compiladors

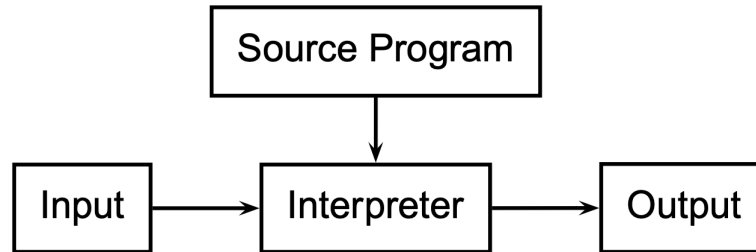
Compilació en C

```
int func(int a, int b) {  
    a = a + b;  
    return a;  
}
```

```
gcc -S prova.c
```

```
_func:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    movl     %edi, -4(%rbp)  
    movl     %esi, -8(%rbp)  
    movl     -4(%rbp), %eax  
    addl     -8(%rbp), %eax  
    movl     %eax, -4(%rbp)  
    movl     -4(%rbp), %eax  
    popq     %rbp  
    retq
```

Intèrprets

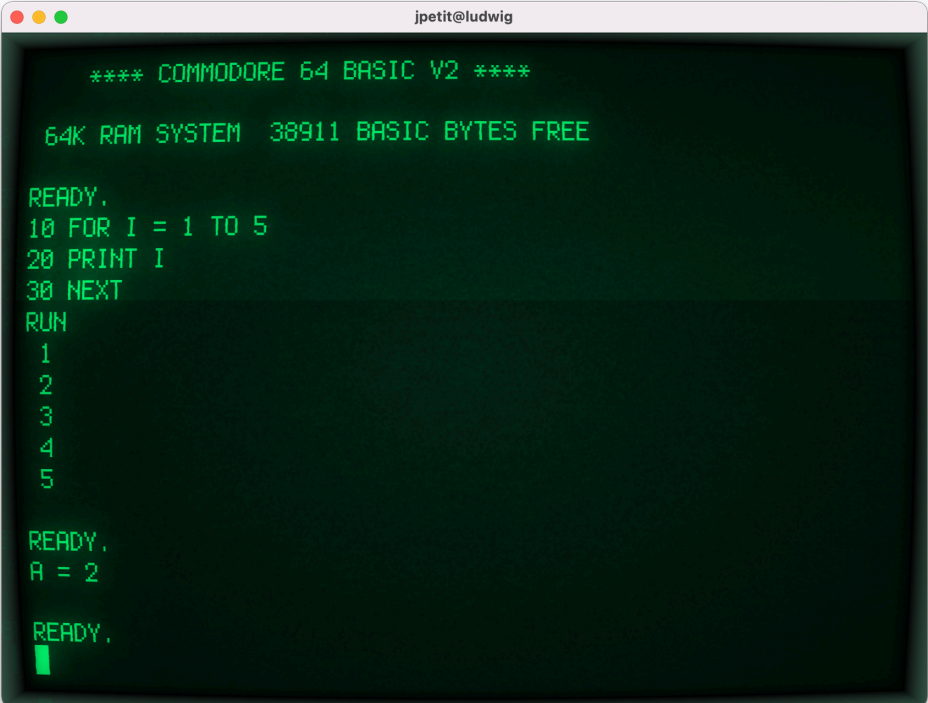


Un **intèrpret** és un programa que executa directament instruccions escrites en un LP.

Exemples: Python, PHP, Perl, ghci, BASIC, Logo...

Intèrprets

Sessió amb l'interpret de BASIC al Commodore 64 (emulat al Mac 🐉!)

A screenshot of a terminal window titled 'jpetit@ludwig' showing the Commodore 64 BASIC V2 interpreter. The window has a black background with green text. The text displayed is: '**** COMMODORE 64 BASIC V2 ****', '64K RAM SYSTEM 38911 BASIC BYTES FREE', 'READY.', '10 FOR I = 1 TO 5', '20 PRINT I', '30 NEXT', 'RUN', '1', '2', '3', '4', '5', 'READY.', 'A = 2', and 'READY.' with a green cursor at the end.

```
jpetit@ludwig

**** COMMODORE 64 BASIC V2 ****

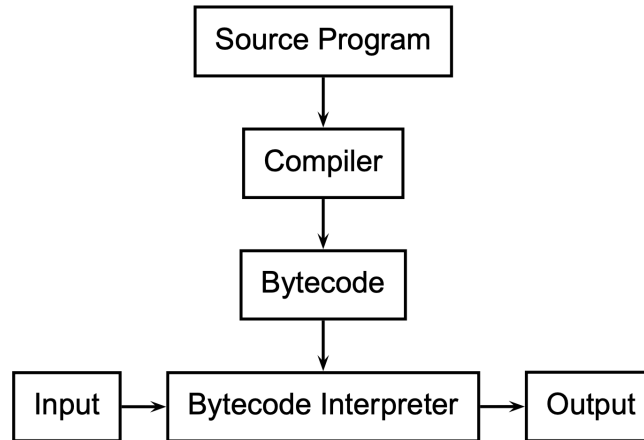
64K RAM SYSTEM 38911 BASIC BYTES FREE

READY.
10 FOR I = 1 TO 5
20 PRINT I
30 NEXT
RUN
1
2
3
4
5

READY.
A = 2

READY.
█
```

Intèrprets de bytecode



Variant entre els compiladors i els intèrprets.

- El **bytecode** és un codi intermedi més abstracte que el codi màquina.
- Augmenta la portabilitat i seguretat i facilita la interpretació.
- Una **màquina virtual** interpreta programes en bytecode.

Exemples: Java, Python, ...

Intèrprets de bytecode

CPython per a Python

```
>>> import dis # desensamblador
>>> dis.dis("a = a + b")
1    0 LOAD_NAME      0 (a)
      2 LOAD_NAME      1 (b)
      4 BINARY_ADD
      6 STORE_NAME      0 (a)
      8 LOAD_CONST      0 (None)
     10 RETURN_VALUE
```

La VM de CPython usa tres tipus de piles:

- **Call stack:** Guarda l'estructura principal de l'execució d'un programa en Python.

Hi ha *frame* per a cada crida oberta a una funció. Cada crida a una funció empila un nou *frame* i cada sortida de funció el desempila. És on es desen les variables locals.

- A cada *frame*, hi ha un **evaluation stack**: És on es fa l'avaluació de les expressions, ficant paràmetres i extraient resultats.
- A cada *frame*, també hi ha un **block stack**: És on es realitza l'execució de les instruccions (condicionals, bucles, `try/excepts`, `withs`, `continues`, `breaks`, ...)

Intèrprets de bytecode

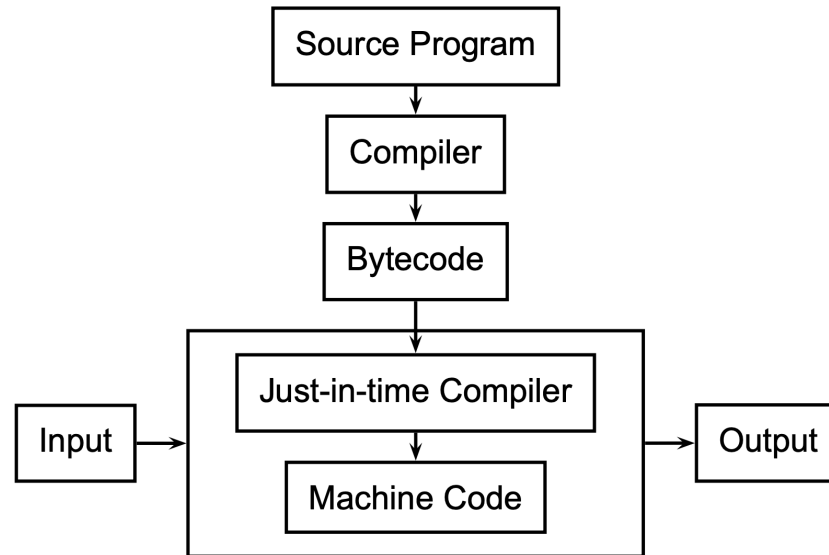
JVM per a Java

```
public static void func(int a, int b) {  
    a = a + b;  
}
```

```
javap -v prova.class
```

```
public static void func(int, int);  
descriptor: (II)V  
flags: (0x0009) ACC_PUBLIC, ACC_STATIC  
Code:  
    stack=2, locals=2, args_size=2  
      0: iload_0  
      1: iload_1  
      2: iadd  
      3: istore_0  
      4: return
```


Compiladors *just-in-time*



La compilació **just-in-time** (JIT) compila fragments del programa durant la seva execució.

Un analitzador inspecciona el codi executat per veure quan val la pena compilar-lo.

Exemples: Julia, V8 per Javascript, JVM per Java, ...

Compiladors *just-in-time*

Numba tradueix funcions en Python a codi màquina i optimitzat usant LLVM en temps d'execució.

```
import numba
import random

@numba.jit(nopython=True)
def monte_carlo_pi(nsamples):
    acc = 0
    for i in range(nsamples):
        x = random.random()
        y = random.random()
        if x*x + y*y < 1.0:
            acc += 1
    return 4.0 * acc / nsamples
```

Preprocessadors

Un **preprocessador** prepara el codi font d'un programa abans que el compilador el vegi.

- Expansió de macros
- Inclusió de fitxers
- Compilació condicional
- Extensions de llenguatge

Exemples: cpp, m4, ...

Preprocessadors

El preprocessador de C

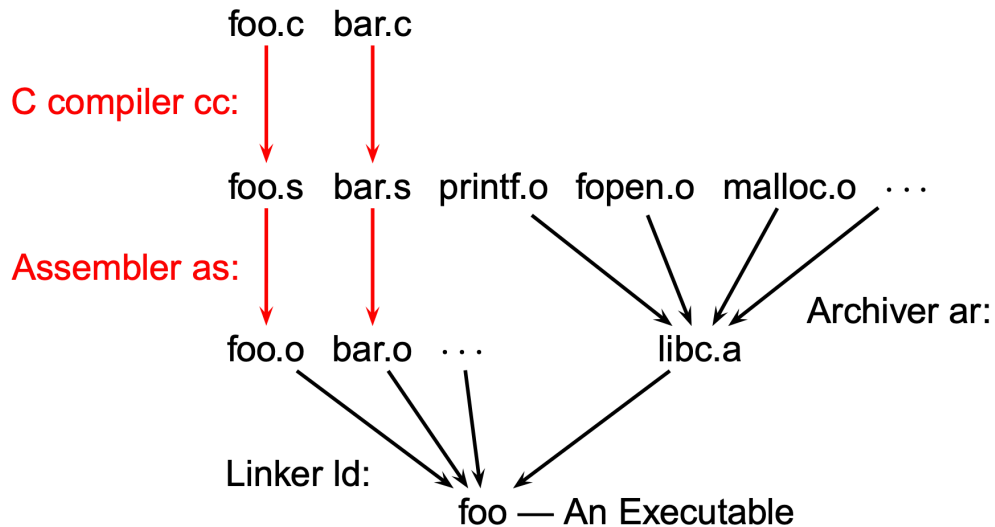
```
#include <stdio.h>
#define min(x, y) ((x)<(y))?(x):(y)
#ifdef DEFINE_BAZ
int baz();
#endif
void foo() {
    int a = 1;
    int b = 2;
    int c;
    c = min(a,b);
}
```

`gcc -E programa.c`

```
extern int printf(char*, ...);
: moltes més línies de stdio.h
void foo() {
    int a = 1;
    int b = 2;
    int c;
    c = ((a)<(b))?(a):(b);
}
```

Ecosistema

Els processadors de llenguatges viuen en un ecosistema gran i complex: preprocessadors, compiladors, enllaçadors, gestors de llibreries, ABIs (application binary interfaces), formats d'executables, ...



Contingut

- Introducció
- Visió general
 - Processadors de llenguatges
 - Sintaxi
 - Semàntica
 - Flux de compilació
 - Eines
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Sintaxi

La **sintaxi** d'un llenguatge de programació és el conjunt de regles que defineixen les combinacions de símbols que es consideren construccions correctament estructurades.

Jove xef, porti whisky amb quinze glaçons d'hidrogen, coi!

→ frase sintàcticament correcta en català, però no és un programa en Java.

```
class Foo {  
    public int j;  
    public int foo(int k) {  
        return j + k;  
    }  
}
```

→ programa sintàcticament correcte en Java, però no és un programa en C.

Sintaxi

Sovint s'especifica la sintaxi utilitzant una **gramàtica lliure de context** (*context-free grammar*).

Els elements més bàsics ("paraules") s'especifiquen a través d'**expressions regulars**.

Exemple per expressions algebraiques:

```
expr → NUM  
      | '(' expr ')'  
      | expr '+' expr  
      | expr '-' expr  
      | expr '*' expr  
      | expr '/' expr  
  
NUM  → [0-9]+ ( '.' [0-9]+ )
```


Contingut

- Introducció
- Visió general
 - Processadors de llenguatges
 - Sintaxi
 - Semàntica
 - Flux de compilació
 - Eines
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Semàntica

La **semàntica** d'un LP descriu què significa un programa ben construït.

```
def fib(n):  
    a, b = 0, 1  
    for i in range(n):  
        a, b = b, a + b  
    return a
```

→ La semàntica d'aquesta funció en Python és el càlcul de l' n -èsim nombre de Fibonacci.

Semàntica

A vegades, les construccions sintàcticament correctes poden ser semànticament incorrectes.

L'**arc** de **Sant Martí** va saltar sobre el planeta pelut.

→ sintàcticament correcta en català, però sense sentit.

```
class Foo {  
    int bar(int x) { return Foo; }  
}
```

→ sintàcticament correcta en Java, però sense sentit.

Semàntica

A vegades, les construccions sintàcticament correctes poden ser ambigües.

Han posat un banc a la plaça.

→ sintàcticament correcta en català, però ambigua

```
class Bar {  
    public float foo() { return 0; }  
    public int foo()   { return 0; }  
}
```

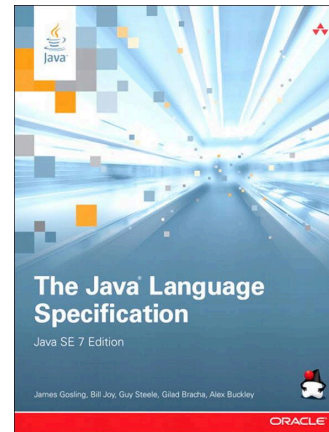
→ sintàcticament correcta en Java, però ambigu.

Semàntica

Hi ha bàsicament dues maneres d'especificar formalment la semàntica:

- **Semàntica operacional:** defineix una màquina virtual i com l'execució del programa canvia l'estat de la màquina.
- **Semàntica denotacional:** mostra com construir una funció que representa el comportament del programa (és a dir, una transformació d'entrades a sortides) a partir de les construccions del LP.

La majoria de definicions de semàntica per a LPs utilitzen una semàntica operacional descrita informalment en llenguatge natural.



Contingut

- Introducció
- Visió general
 - Processadors de llenguatges
 - Sintaxi
 - Semàntica
 - Flux de compilació
 - Eines
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Etapas

- Front end
 - preprocessador
 - analitzador lèxic (escàner)
 - analitzador sintàctic (parser)
 - analitzador semàntic
- Middle end
 - generador de codi intermedi
 - optimitzador de codi intermedi
- Back end
 - generador de codi específic
 - optimitzador de codi específic

Entrada

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a -= b; else b -= a;  
    }  
    return a;  
}
```

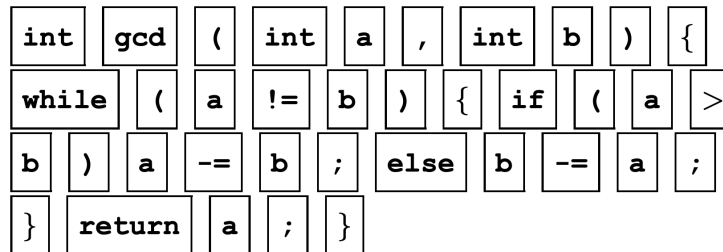
El compilador veu una seqüència de caràcters:

```
i n t _ g c d ( i n t _ a , _ i n t _ b ) _ {   
( a _ ! = _ b ) _ {   
; _ e l s e _ b _ - = _ a ;   
_ r e t u r n _ a ; }
```


Analitzador lèxic

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a -= b; else b -= a;  
    }  
    return a;  
}
```

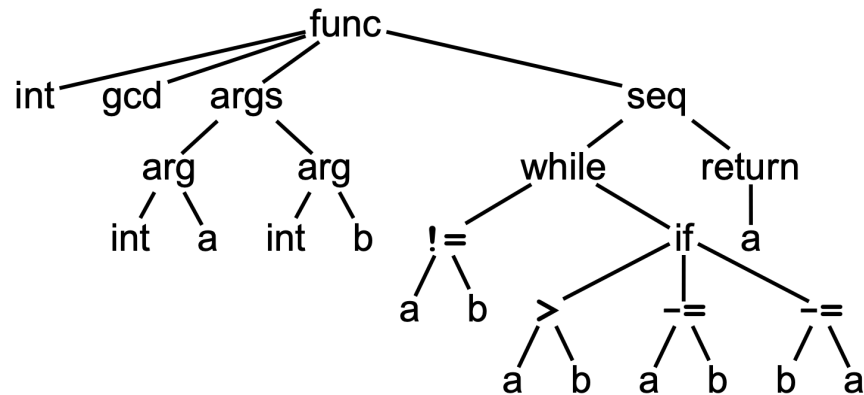
L'analitzador lèxic (escàner) agrupa els caràcters en "paraules" (tokens) i elimina blancs i comentaris.



Analitzador sintàctic

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a -= b; else b -= a;  
    }  
    return a;  
}
```

L'analitzador sintàctic (**parser**) construeix un **arbre de sintaxi abstracta** (AST) a partir de la seqüència de tokens i les regles sintàctiques.



Les paraules clau, els separadors, parèntesis i blocs s'eliminen.

Generador de codi intermedi

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a -= b; else b -= a;  
    }  
    return a;  
}
```

El **generador de codi** tradueix el programa a **codi de tres adreces** (ensamblador idealitzat amb infinitat de registres).

```
L0:    sne $1, a, b           # signed not equal  
       seq $0, $1, 0         # signed equal  
       btrue $0, L1          # while a != b  
       sl $3, b, a           # signed less  
       seq $2, $3, 0  
       btrue $2, L4          # if a < b  
       sub a, a, b           # a -= b  
       jmp L5  
L4:    sub b, b, a           # b -= a  
L5:    jmp L0  
L1:    ret a                 # return a
```

Optimitzador de codi intermedi

```
L0:    sne $1, a, b           # signed not equal
      seq $0, $1, 0         # signed equal
      btrue $0, L1          # while a != b
      sl $3, b, a           # signed less
      seq $2, $3, 0
      btrue $2, L4          # if a < b
      sub a, a, b           # a -= b
      jmp L5
L4:    sub b, b, a           # b -= a
L5:    jmp L0
L1:    ret a                # return a
```

El **back end** tradueix i optimitza el codi de tres adreces a l'arquitectura desitjada:

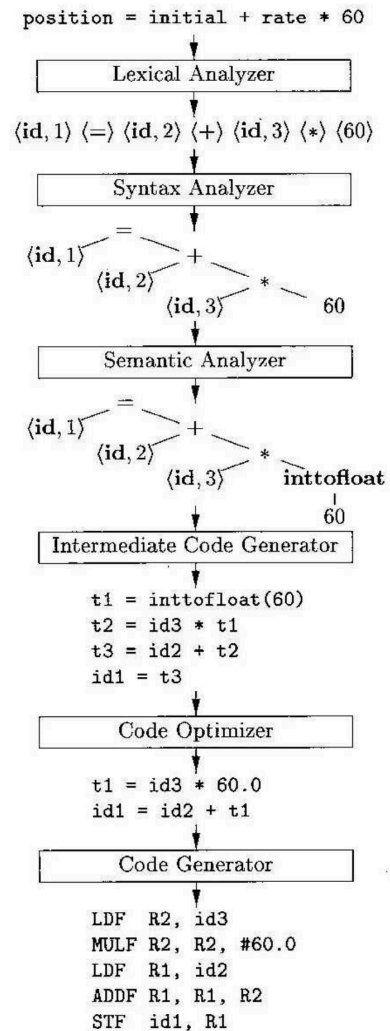
```
gcd:   pushl %ebp           # Save FP [asm 80386]
      movl %esp,%ebp
      movl 8(%ebp),%eax     # Load a from stack
      movl 12(%ebp),%edx    # Load b from stack
.L8:   cmpl %edx,%eax
      je .L3               # while a != b
      jle .L5              # if (a < b)
      subl %edx,%eax       # a -= b
      jmp .L8
.L5:   subl %eax,%edx       # b -= a
      jmp .L8
.L3:   leave               # Restore SP, BP
      ret                 # return a
```

Sumari

- 1
- 2
- 3

position	...
initial	...
rate	...

SYMBOL TABLE



Contingut

- Introducció
- Visió general
 - Processadors de llenguatges
 - Sintaxi
 - Semàntica
 - Flux de compilació
 - Eines
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Eines

Per construir un compilador no es parteix de zero.

Hi ha moltes eines que donen suport.

Exemples:

- ANTLR, donades les especificacions lèxiques i sintàctiques del LP, construeix automàticament l'escàner, l'analitzador i l'AST.
- LLVM ofereix una col·lecció d'eines modulars reutilitzables pels backends dels compiladors.

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Anàlisi lèxica

L'**analitzador lèxic** (o **escàner**) converteix una seqüència de caràcters en una seqüència de **tokens**:

- identificadors,
- literals (nombres, textos, caràcters)
- paraules clau,
- operadors,
- puntuació...

`f o o _ = _ a + _ bar (2, _ q) ;`



Token	Lexemes	Pattern
EQUALS	=	an equals sign
PLUS	+	a plus sign
ID	a foo bar	letter followed by letters or digits
NUM	0 42	one or more digits

Objectius

- Simplificar la feina de l'analitzador sintàctic.

El parser no té en compte els noms dels identificadors, només li preocupen els tokens (`supercalifragilisticexpialidocious` → `ID`).

- Descartar detalls irrelevants: blancs, comentaris, ...
- Els escàners són molt més ràpids que els parsers.

Descripció de tokens

Per especificar els tokens s'utilitzen conceptes de la teoria de llenguatges:

Alfabet: un conjunt finit de símbols.

Exemples: $\{0, 1\}$, $\{A, B, \dots, Z\}$, ASCII, Unicode, ...

Paraula: una seqüència finita de símbols de l'alfabet.

Exemples: ϵ (la paraula buida), `foo`, $\alpha\beta\gamma$.

Llenguatge: Un conjunt de paraules sobre un alfabet.

Exemples: $\emptyset$ (el llenguatge buit), $\{1, 11, 111, 1111\}$, tots els mots anglesos, tots els identificadors (textos que comencen amb una lletra seguida per lletres o dígitos).

Operacions sobre llenguatges

Exemple: $L = \{ \varepsilon, wo \}$, $M = \{ man, men \}$

Concatenació: Una paraules d'un llenguatge seguida d'una paraula de l'altre llenguatge.

$LM = \{ man, men, woman, women \}$

Unió: Totes les paraules de cada llenguatge.

$L \cup M = \{ \varepsilon, wo, man, men \}$

Clausura de Kleene: Zero o més concatenacions.

$M^* = \{ \varepsilon \} \cup M \cup MM \cup MMM \cup \dots = \{ \varepsilon, man, men, manman, manmen, menman, menmen, manmanman, manmanmen, manmenman, \dots \}$

Expressions regulars

Les **expressions regulars** descriuen llenguatges a partir de tokens sobre un alfabet Σ .

1. ϵ és una expressió regular que denota $\{\epsilon\}$.
2. Si $a \in \Sigma$, a és una expressió regular que denota $\{a\}$.
1. Si r i s denoten els llenguatges $L(r)$ i $L(s)$,
 - $(r) | (s)$ denota $L(r) \cup L(s)$
 - $(r)(s)$ denota $\{tu : t \in L(r), u \in L(s)\}$
 - (r^*) denota la clausura transitiva de $L(r)$

Expressions regulars

Exemples:

$$\Sigma = \{a, b\}$$

RE	Language
$a b$	$\{a, b\}$
$(a b)(a b)$	$\{aa, ab, ba, bb\}$
a^*	$\{\epsilon, a, aa, aaa, aaaa, \dots\}$
$(a b)^*$	$\{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \dots\}$
$a a^*b$	$\{a, b, ab, aab, aaab, aaaab, \dots\}$

Generadors d'escàners

Les expressions regulars s'usen en eines per crear compiladors:

- lex, ANTLR, ...

I també,

- en comandes del SO per tractar fitxers (grep, sed, ...)
- en llibreries d'LPs per tractar textos (re en Python, directament en Javascript, ...)

Generadors d'escàners

A partir de la definició lèxica, l'escàner és un autòmat determinista que produeix com a sortida els tokens reconeguts.

Construcció:

Regles amb expressions regulars



Autòmats finits no deterministes



Autòmat finit determinista



Taules

Analitzador lèxic d'ANTLR

```
DIGIT   : '0'..'9' ;  
LETTER  : [a-zA-Z] ;  
  
NUMBER  : (DIGIT)+ ;  
IDENT   : LETTER (LETTER | DIGIT)* ;  
  
WS      : [ \t\n]+ -> skip ;
```

Referència

- els noms dels tokens han de ser en majúscules
- textos entre cometes representen aquell text
- `ajuntar` indica concatenació
- `|` indica unió
- `*` indica zero o més
- `+` indica un o més
- `?` indica un o cap
- `..` indica rangs
- es poden agrupar elements amb parèntesis
- l'`skip` no reporta el token

grep

La comanda `grep` (*global regular expression print*) permet cercar patrons en texts.

```
grep 'jpetit' /etc/passwd
```

Imatge: Dave Child, cheatography.com

Anchors	Assertions	Groups and Ranges
<code>^</code> Start of string, or start of line in multi-line pattern	<code>?=</code> Lookahead assertion	<code>.</code> Any character except new line (<code>\n</code>)
<code>\A</code> Start of string	<code>?!</code> Negative lookahead	<code>(a b)</code> a or b
<code>\$</code> End of string, or end of line in multi-line pattern	<code>?<=</code> Lookbehind assertion	<code>(...)</code> Group
<code>\Z</code> End of string	<code>?!= or ?<! </code> Negative lookbehind	<code>(?....)</code> Passive (non-capturing) group
<code>\b</code> Word boundary	<code>?></code> Once-only Subexpression	<code>[abc]</code> Range (a or b or c)
<code>\B</code> Not word boundary	<code>?()</code> Condition [if then]	<code>[^abc]</code> Not (a or b or c)
<code>\<</code> Start of word	<code>?() </code> Condition [if then else]	<code>[a-q]</code> Lower case letter from a to q
<code>\></code> End of word	<code>?#</code> Comment	<code>[A-Q]</code> Upper case letter from A to Q
Character Classes	Quantifiers	<code>[0-7]</code> Digit from 0 to 7
<code>\c</code> Control character	<code>*</code> 0 or more {3} Exactly 3	<code>\x</code> Group/subpattern number "x"
<code>\s</code> White space	<code>+</code> 1 or more {3,} 3 or more	Ranges are inclusive.
<code>\S</code> Not white space	<code>?</code> 0 or 1 {3,5} 3, 4 or 5	Pattern Modifiers
<code>\d</code> Digit	Add a <code>?</code> to a quantifier to make it ungreedy.	<code>g</code> Global match
<code>\D</code> Not digit	Escape Sequences	<code>i *</code> Case-insensitive
<code>\w</code> Word	<code>\</code> Escape following character	<code>m *</code> Multiple lines
<code>\W</code> Not word	<code>\Q</code> Begin literal sequence	<code>s *</code> Treat string as single line
<code>\x</code> Hexadecimal digit	<code>\E</code> End literal sequence	<code>x *</code> Allow comments and whitespace in pattern
<code>\O</code> Octal digit	"Escaping" is a way of treating characters which have a special meaning in regular expressions literally, rather than as special characters.	<code>e *</code> Evaluate replacement
POSIX		<code>U *</code> Ungreedy pattern
<code>[upper:]</code> Upper case letters	Common Metacharacters	<code>*</code> PCRE modifier
<code>[lower:]</code> Lower case letters	<code>^</code> [. \$	String Replacement
<code>[alpha:]</code> All letters	{ * (\	<code>\$n</code> nth non-passive group
<code>[alnum:]</code> Digits and letters	+) ?	<code>\$2</code> "xyz" in /^(abc)(xyz)\$/
<code>[digit:]</code> Digits	< >	<code>\$1</code> "xyz" in /^(?:abc)(xyz)\$/
<code>[xdigit:]</code> Hexadecimal digits	The escape character is usually <code>\</code>	<code>\$'</code> Before matched string
<code>[punct:]</code> Punctuation	Special Characters	<code>\$'</code> After matched string
<code>[blank:]</code> Space and tab	<code>\n</code> New line	<code>\$+</code> Last matched string
<code>[space:]</code> Blank characters	<code>\r</code> Carriage return	<code>\$&</code> Entire matched string
<code>[cntrl:]</code> Control characters	<code>\t</code> Tab	Some regex implementations use <code>\</code> instead of <code>\$</code> .
<code>[graph:]</code> Printed characters	<code>\v</code> Vertical tab	
<code>[print:]</code> Printed characters and spaces	<code>\f</code> Form feed	
<code>[word:]</code> Digits, letters and underscore	<code>\xxx</code> Octal character xxx	
	<code>\hhh</code> Hex character hh	

re

El mòdul `re` de Python proporciona operacions sobre expressions regulars.

```
>>> import re

>>> p = re.compile('[a-z]+')
>>> p.match("7654386732().")
None
>>> m = p.match('unicorn')
>>> m.group()
'unicorn'
>>> m.start(), m.end()
(0, 7)
>>> m.span()
(0, 7)

>>> re.search('casa', 'vaig a casa a dormir')
<re.Match object; span=(7, 11), match='casa'>

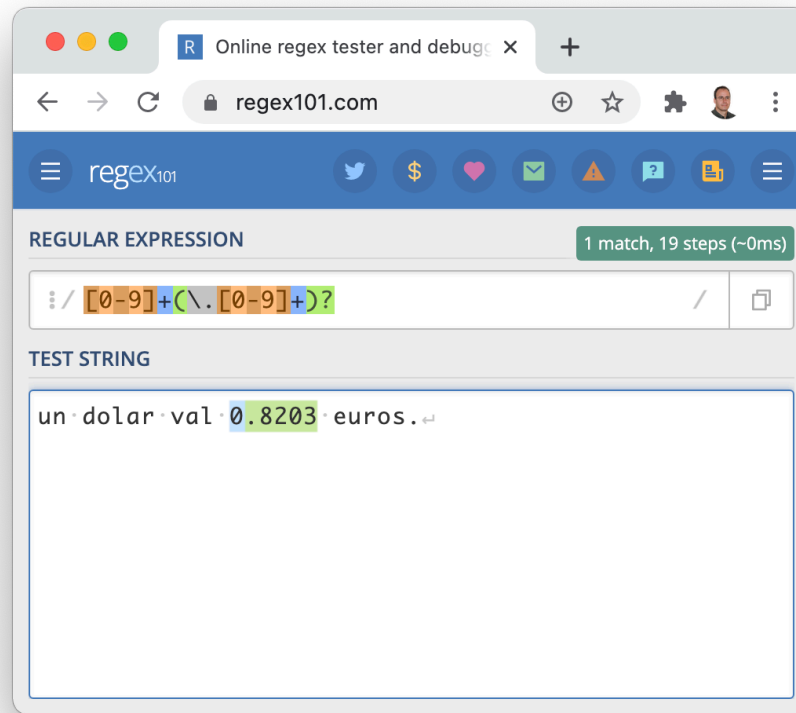
>>> re.sub('[0-9]+', 'molts', 'val 350 euros')
'val molts euros'
```

Tutorial

Testing d'expressions regulars

Some people, when confronted with a problem, think "*I'll use regular expressions*". Now they have two problems.

— Jamie Zawinski, 1997



Exercicis

Definiu expressions regulars pels llenguatges següents:

1. Identificadors de C++: poden contenir lletres, dígits i subratllats però no poden començar per un dígit.
2. Nombres en coma flotant. Si s'utilitza un punt decimal, un dígit decimal és obligatori.

Exemples: `3.1416`, `-3e4`, `+1.0e-5`, `.567e+8`, ...

3. Totes les paraules amb alfabet `{a, b, c}` en les quals la primera aparició de `b` sempre és precedida per, com a mínim, una aparició de `a`.
4. Totes les paraules amb minúscules que contenen les cinc vocals en ordre (cada vocal només pot aparèixer un sol cop).

Example: `zfaehipojksuj`

5. Totes les paraules amb minúscules en les quals les lletres es troben en ordre lexicogràfic ascendent.

Examples: `afhmnqsyx`, `abcdz`, `dgky`, ... Contraexemple: `bdeaz`.

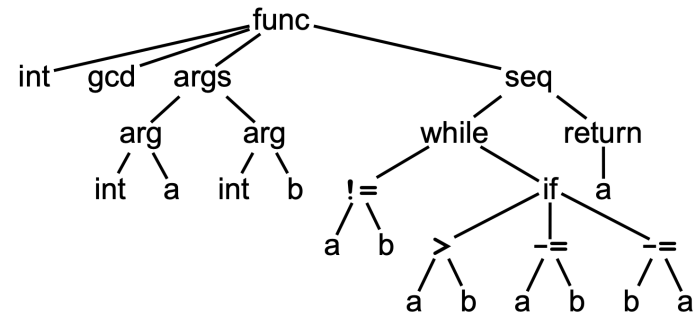
Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
 - Desambiguació de gramàtiques
 - Generadors d'analitzadors
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Anàlisi sintàctica

L'objectiu de l'**analitzador sintàctic** (o **parser**) és convertir una seqüència de tokens en un arbre de sintaxi abstracta que capturi la jerarquia de les construccions.

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a -= b; else b -= a;  
    }  
    return a;  
}
```



- Es descarta informació no rellevant com les paraules clau, els separadors, els parèntesis i els blocs.
- Es facilita la feina dels propers estadis.

Gramàtiques

La majoria dels LPs es descriuen a través de **gramàtiques incontextuals**, usant notació **BNF** (Backus–Naur form).

```
pgma → expr ; pgma
      | ε

expr  → expr + expr
      | expr - expr
      | expr * expr
      | expr / expr
      | ( expr )
      | NUM
```

Les gramàtiques incontextuals permeten descriure llenguatges més amplis que els llenguatges regulars perquè són "recursives".

Exemple: Llenguatge dels mots capicues

✓ gramàtica incontextual

✗ expressió regular

→ La recursivitat permet definir jerarquies i niuar elements (parèntesis o blocs).

Gramàtiques

Exemple: Gramàtica de C 

```
translation-unit      : {external-declaration}*  
external-declaration  : function-definition  
                      | declaration  
function-definition   : {declaration-specifier}* declarator {declaration}* compound-  
declaration-specifier : storage-class-specifier  
                      | type-specifier  
                      | type-qualifier  
storage-class-specifier : auto  
                      | register  
                      | static  
                      | extern  
                      | typedef  
type-specifier         : void  
                      | char  
                      | short  
                      | int  
                      | long  
                      | float  
                      | double  
                      | signed  
                      | unsigned  
                      | struct-or-union-specifier  
                      | enum-specifier
```

Dificultats

- Gramàtiques ambigües
- Prioritat dels operadors
- Associativitat dels operadors
- Recursivitat per la dreta / per l'esquerra

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
 - Desambiguació de gramàtiques
 - Generadors d'analitzadors
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

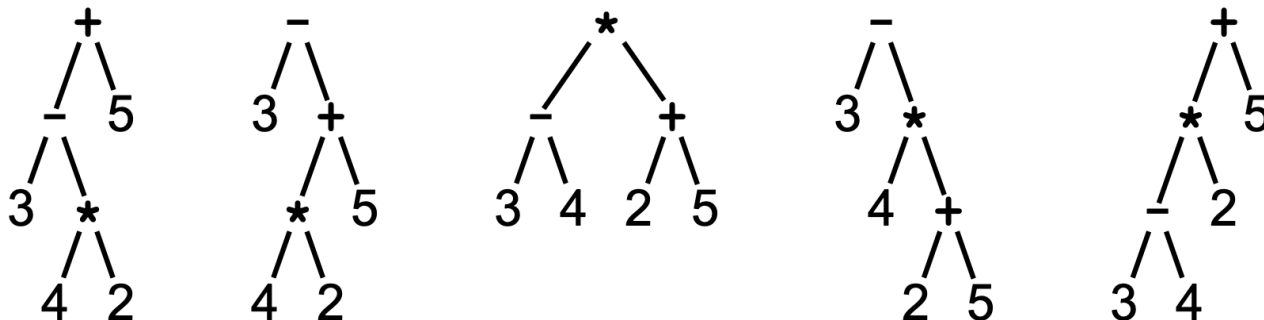
Gramàtiques ambigües

Una gramàtica és **ambigua** si un mateix text es pot **derivar** (organitzar en un arbre segons la gramàtica) de diferents maneres.

Per exemple, amb

expr $\rightarrow$ expr + expr | expr - expr | expr * expr | **NUM**

el fragment 3 - 4 * 2 + 5 es pot derivar d'aquestes maneres:

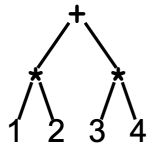


Gramàtiques ambigües

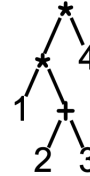
La prioritat i associativitat permeten eliminar ambigüitats.

- Prioritat: $1 * 2 + 3 * 4$

$*$ té més prioritat que $+$

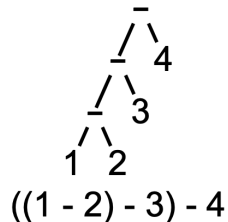


$+$ té més prioritat que $*$

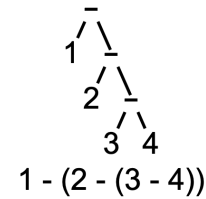


- Associativitat: $1 - 2 - 3 - 4$

associativitat per l'esquerra



associativitat per la dreta



Desambiguació clàssica

Comencem amb:

```
expr → expr + expr  
      | expr - expr  
      | expr * expr  
      | expr / expr  
      | NUM
```

➡ És ambigua: no hi ha la prioritat ni associativitat.

Desambiguació clàssica

Podem assignar prioritats trencant en vàries regles, una per nivell:

```
expr → expr + expr  
      | expr - expr  
      | expr * expr  
      | expr / expr  
      | NUM
```



```
expr → expr + expr  
      | expr - expr  
      | term  
  
term → term * term  
      | term / term  
      | NUM
```

→ Encara és ambigua: falta associativitat.

Desambiguació clàssica

Podem fer que un costat o altre afecti el següent nivell de prioritat:

```
expr → expr + expr  
      | expr - expr  
      | term
```

```
term → term * term  
      | term / term  
      | NUM
```



```
expr → expr + term  
      | expr - term  
      | term
```

```
term → term * NUM  
      | term / NUM  
      | NUM
```

✓ La gramàtica ja no és ambigua.

✗ Però el - queda associat per la dreta...

Desambiguació amb ANTLR

- La prioritat de les regles ve donada per l'ordre d'escriptura
- Podem combinar components en la mateixa regla
- L'associativitat és per l'esquerra per defecte
- Té una manera de definir l'associativitat per la dreta

Exemple:

```
expr : <assoc=right> expr '^' expr
    | expr ('*' | '/') expr
    | expr ('+' | '-') expr
    | NUM ;
```

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
 - Desambiguació de gramàtiques
 - Generadors d'analitzadors
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Generadors d'analitzadors sintàctics

Existeixen diferents tècniques per generar analitzadors sintàctics, amb propietats diferents.

- **Analitzadors descendents** (*top-down parsers*): reconeixen l'entrada d'esquerra a dreta tot buscant derivacions per l'esquerra expandint la gramàtica de l'arrel cap a les fulles.
- **Analitzadors ascendents** (*bottom-up parsers*): reconeixen primer sobre les unitats més petites de l'entrada analitzada abans de reconèixer l'estructura sintàctica segons la gramàtica.

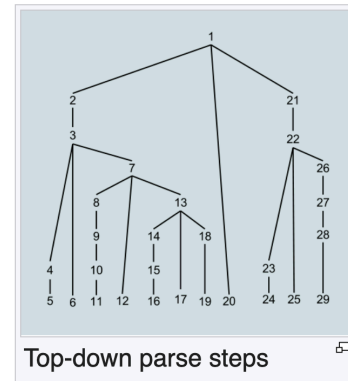
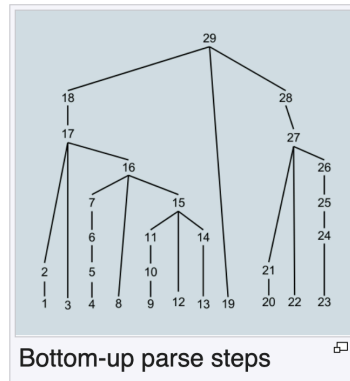
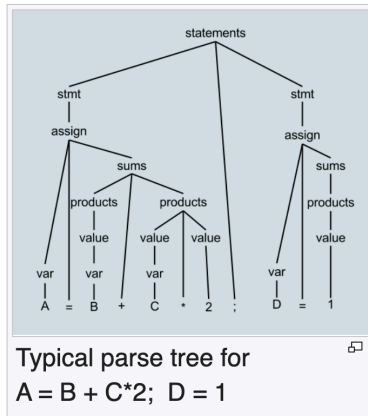


Figura: [Wikipedia](#)

Generadors d'analitzadors sintàctics

Analitzadors descendents $LL(k)$

- LL: Left-to-right, Left-most derivation
- k : nombre de tokens que mira endavant

Idea bàsica: mirar el següent token per poder decidir quina producció utilitzar.

Implementació: Associar una funció a cada construcció del LP.

- Si la construcció està definida per una única regla:

La seva funció associada cridarà a les funcions associades a les construccions que apareixen en aquesta regla i comprovarà que els tokens que apareixen en la definició son els que apareixen en la seqüència donada.

- Si la construcció està definida per diverses regles:

La decisió sobre quina regla s'ha d'aplicar es basa en mirar els següents k tokens.

Generadors d'analitzadors sintàctics

Analitzadors descendents LL(1)

Exemple de gramàtica:

```
root      : stmt* 'end'
           ;

stmt      : 'if' expr 'then' stmt
           | 'while' expr 'do' stmt
           | expr ':= ' expr
           ;

expr      : NUMBER
           | '(' expr ')'
           ;
```

Exercici:

Implementeu `expr()` i `root()`.

Parser LL(1):

```
def stmt():
    if current_token() == IF:
        match(IF)
        cond = expr()
        match(THEN)
        then = stmt()
        return Node(IF, cond, then)
    elif current_token() == WHILE:
        match(WHILE)
        cond = expr()
        match(DO)
        loop = stmt()
        return Node(WHILE, cond, loop)
    elif current_token() in [NUMBER, LPAR]:
        lvalue = expr()
        match(ASSIGN)
        rvalue = expr()
        return Node(ASSIGN, lvalue, rvalue)
    else:
        SyntaxError()
```

Generadors d'analitzadors sintàctics

Analitzadors descendents LL(1)

Inconvenients principals:

- Les produccions no poden tenir prefixos comuns (no sabria quina triar).

```
expr → ID ( expr )  
      | ID = expr
```

- Les regles no poden tenir recursivitat per l'esquerra (es penjaria).

```
expr → expr + term  
      | term
```

Generadors d'analitzadors sintàctics

Analitzadors descendents LL(1)

Comencem amb:

```
expr → expr '+' term  
      | expr '-' term  
      | term
```

 prefixos comuns

↓ factoritzem els prefixos comuns

```
expr → expr ('+' term | '-' term)  
      | term
```

 recursivitat per l'esquerra

↓ substituïm recursivitat per l'esquerra per recursivitat per la dreta

```
expr → term expr2  
expr2 → '+' term expr2  
      | '-' term expr2  
      | ε
```



ANTLR

- ANTLR és un analitzador descendent LL(k).
- Permet explicitar tokens a les regles sintàctiques:

```
expr : expr '+' expr ;
```

- Permet usar * i + a les regles sintàctiques:

```
block : statement+ ;
```

- Permet definir fàcilment la prioritats d'operadors i associativitat:

```
expr : <assoc=right> expr '^' expr  
    | expr ('*' | '/') expr  
    | expr ('+' | '-') expr  
    | NUM ;
```

Exercicis

P1: Escriuiu gramàtiques no ambigües pels llenguatges següents:

1. El conjunt de tots els mots amb `a`s i `b`s que són palíndroms.
2. Mots que tenen el patró `a*b*` amb més `a`s que `b`s.
3. Textos amb parèntesis i claudàtors ben aniuats.
Exemple: `( [ [ ] ( ( ) [ ( ) ] [ ] ) ] )`.
4. El conjunt de tots els mots amb `a`s i `b`s tals que a cada `a` li segueix immediatament per, almenys, una `b`.
5. El conjunt de tots els mots amb `a`s i `b`s amb el mateix nombre d'`a`s que `b`s.
6. El conjunt de tots els mots amb `a`s i `b`s amb un nombre diferent d'`a`s que `b`s.
7. Blocs d'instruccions separades per `;` a la Pascal. Exemple:
`BEGIN instrucció ; BEGIN instrucció ; instrucció END ; instrucció END`.
8. Blocs d'instruccions acabades per `;` a la C.
Exemple: `{ instrucció ; { instrucció ; instrucció ; } ; instrucció ; }`.

P2: Especifiqueu les gramàtiques anteriors amb notació ANTLR, modificant la gramàtica si és necessari.

Exercicis

P3: Sense utilitzar cap eina ni llibreria (ni `eval!`), escriviu en Haskell, Python o C++ un analitzador descendent LL(1) que llegeixi una seqüència d'expressions i escrigui el resultat de cadascuna d'elles.
[TBD: problema pel Jutge! 😊]

- Entrada:

```
2
2 * 3
2 * 3 + 1
2 * (3 + 1)
10 - 3 - 2
13 / 3
```

- Sortida:

```
2
6
7
8
5
4
```

Exercicis

P4: Sense utilitzar cap eina ni llibreria, escriviu en Haskell, Python o C++ un analitzador descendent LL(1) que llegeixi una seqüència d'expressions i construeixi i escrigui l'arbre de sintaxi abstracta de cadascuna. [TBD: problema pel Jutge! 😊]

- Entrada:

```
2 * (3 + 1)
(10 - 3 - 2) / 4
```

- Sortida:

```
(MUL 2 (SUM 3 1))
(DIV (SUB (SUB 10 3) 2) 4)
```

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Accions

- Amb un analitzador descendent, es poden executar accions durant el reconeixement de les regles.

Les accions poden aparèixer en qualsevol punt de la regla:

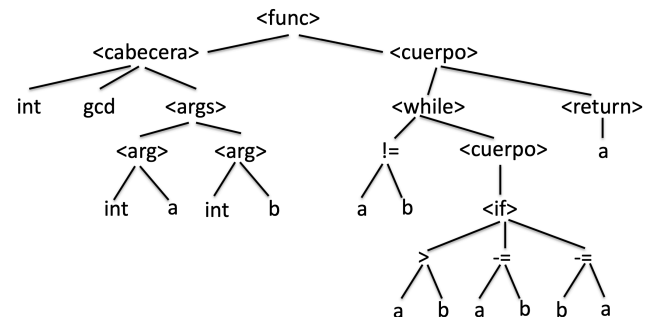
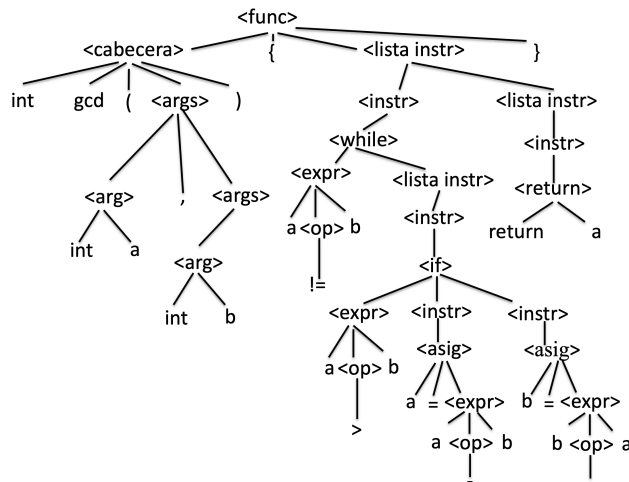
```
regla    :    { /* abans */    }
           regla1
           { /* durant */    }
           regla2
           { /* durant */    }
           regla3
           { /* després */    }
           ;
```

- La gramàtica esdevé "imperativa".
 - Les accions s'entrellacen amb la gramàtica.
 - És fàcil entendre què passa i quan passa.
- Amb un analitzador ascendent, només es poden executar accions després de reconèixer una regla.

Arbres de sintaxi

Usualment, les accions construeixen un arbre de sintaxi concreta que segueix les regles de la gramàtica.

Aquest arbre es sol convertir en un arbre de sintaxi abstracta.



- Es separa l'anàlisi de la traducció.
- Es facilita les modificacions tot minimitzant les interaccions.
- Es permet que diferents parts del programa s'analitzin en ordres diferents.

Arbres de sintaxi concreta vs abstracta

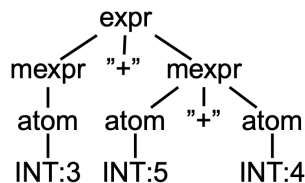
Arbre de sintaxi concreta / de derivació: Reflecteix exactament les regles sintàctiques.

Arbre de sintaxi abstracta (*abstract syntax tree*, AST): Representa el programa fidelment, però elimina i simplifica detalls sintàctics irrelevants.

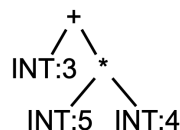
Exemple: Eliminar regles per desambiguar gramàtica.

```
expr    : mexpr ( '+' mexpr ) * ;  
mexpr   : atom  ( '*' atom  ) * ;  
atom    : NUM  ;
```

3 + 5 * 4



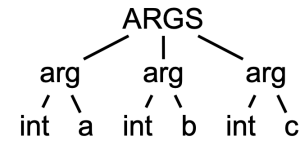
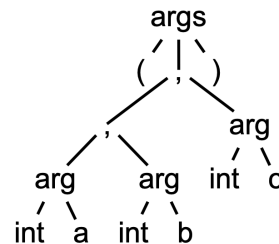
Concrete Parse Tree



Abstract Syntax Tree

Exemple: Aplanar una llista de paràmetres.

```
int gcd(int a, int b, int c)
```



Ús dels ASTs

Un cop construït l'AST, les etapes següents el recorren per a dur a terme les seves tasques:

- L'anàlisi semàntica verificarà l'ús correcte dels elements del programa.
- El generador de codi visitarà l'arbre i li aplicarà regles per generar codi intermedi.
- L'interpret es passejarà per l'arbre per dur a terme les seves instruccions.

ASTs en ANTLR

A partir de la gramàtica, ANTLR pot generar un analitzador descendent amb accions que construeixin un AST.

L'AST es pot visitar a través de *visitors* (un patró de disseny).

ANTLR també genera la interfície dels visitadors, tot generant un esquelet de mètodes que podem heretar.

Cada mètode s'aplica sobre un tipus de node que correspon a cada regla de la gramàtica.

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Anàlisi semàntica

L'analitzador semàntic recorre l'AST, per obtenir tota la informació necessària per poder generar codi.

Objectius:

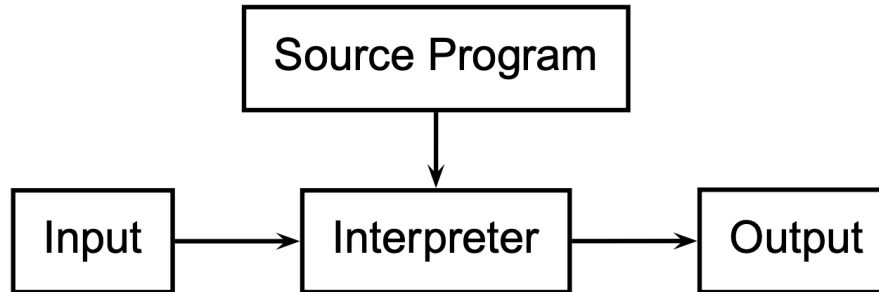
- Comprovar la correcció semàntica del programa (comprovació de tipus).
- Resoldre ambigüitats.
- Assignar memòria.
- Construir la taula de símbols.

— En aquest curs no entrem en aquest vast tema, que deixem per l'assignatura de Compiladors (recomanada!). Però al tema "Inferència de tipus" veure com fer comprovació de tipus.

Contingut

- Introducció
- Visió general
- Anàlisi lèxica
- Anàlisi sintàctica
- Arbres de sintaxi abstracta
- Anàlisi semàntica
- Interpretació

Interpretació



Tutorial: Escriure (en Haskell) un intèrpret per a l'AST d'un senzill llenguatge de programació (SimpleLP™).

SimpleLP

Definició del llenguatge

- Tipus de dades: enters.
- Variables: sempre visibles, inicialitzades a 0.
- Operadors aritmètics: $+$ i $-$.
- Operadors relacionals: $\neq$ i $<$ (retornen 0 o 1).
- Instruccions: assignació, composició seqüencial, condicional i iteració.

Exemple

```
# Algorisme d'Euclides per calcular el mcd de 105 i 252.  
a  $\leftarrow$  105  
b  $\leftarrow$  252  
while  $a \neq b$  do  
  if  $a < b$  then  
     $b \leftarrow b - a$   
  else  
     $a \leftarrow a - b$   
  end  
end
```

SimpleLP

Tipus de dades per l'AST

```
data Expr
= Val Int           -- valor
| Var String        -- variable
| Add Expr Expr     -- suma (+)
| Sub Expr Expr     -- resta (-)
| Neq Expr Expr     -- diferent-de (≠)
| Lth Expr Expr     -- menor-que (<)

data Instr
= Ass String Expr   -- assignació
| Seq [Instr]        -- composició seqüencial
| Cond Expr Instr Instr -- condicional
| Loop Expr Instr    -- iteració
```


SimpleLP

Tipus de dades per l'AST

Exemple: AST pel programa anterior:

```
Seq [  
  (Ass "a" (Val 105))  
  ,  
  (Ass "b" (Val 252))  
  ,  
  (While  
    (Neq (Var "a") (Var "b"))  
    (Cond  
      (Lth (Var "a") (Var "b"))  
      (Ass "b" (Sub (Var "b") (Var "a")))  
      (Ass "a" (Sub (Var "a") (Var "b")))  
    )  
  )  
]
```

SimpleLP

Memòria

Descrivim el valor de les variables a través d'una memòria de tipus `Mem` amb aquestes operacions:

```
-- retorna una memòria buida
empty :: Mem

-- insereix (o canvia si ja hi era) una clau amb el seu valor
update :: Mem -> String -> Int -> Mem

-- consulta el valor d'una clau en una memòria
search :: Mem -> String -> Maybe Int

-- retorna la llista de claus en una memòria
keys :: Mem -> [String]
```

La implementació seria amb qualsevol diccionari (BST, AVL, hashing, ...).
[Si voleu provar-ho, useu `Map` de `Data.Map`.]

SimpleLP

Avaluació de les expressions

```
--- avalua una expressió en un estat de la memòria
eval :: Expr -> Mem -> Int

eval (Val x) m = x
eval (Var v) m =
    case search v m of
        Nothing -> 0
        Just x   -> x
eval (Add e1 e2) m = eval' e1 e2 m (+)
eval (Sub e1 e2) m = eval' e1 e2 m (-)
eval (Neq e1 e2) m = b2i $ eval' e1 e2 m (/=)
eval (Lth e1 e2) m = b2i $ eval' e1 e2 m (<)

eval' e1 e2 m op = op (eval e1 m) (eval e2 m)

b2i False = 0
b2i True  = 1
```

SimpleLP

Interpretació de les instruccions

```
--- retorna l'estat final de la memòria després d'executar una instrucció
--- partint d'un estat inicial de la memòria
exec :: Instr -> Mem -> Mem

exec (Ass v e) m = update v (eval e m) m
exec (Seq []) m = m
exec (Seq (i:is)) m = exec (Seq is) (exec i m)
exec (Cond b i1 i2) m = exec (if eval b m /= 0 then i1 else i2) m
exec (Loop b i) m =
    if eval b m /= 0
    then exec (Loop b i) (exec i m)
    else m
```

SimpleLP

Execució del programa

Escriu el valor final de cada variable (en ordre lexicogràfic) després d'executar una instrucció partint d'una memòria buida.

```
run :: Instr -> IO ()

run i = mapM_ printEntry $ sort $ keys m
  where
    m = exec i empty
    printEntry k = do
      putStr k
      putStr " : "
      print $ fromJust $ search k m
```

Execució amb el programa anterior:

```
a : 21
b : 21
```

Exercicis

1. Modifiquen l'AST i `eval` per tenir operadors de resta i producte.
2. Modifiquen l'AST, `exec` i `run` per tal d'afegir a SimpleLP una nova instrucció `print` que escriui el contingut d'una expressió. Ara `run` només ha d'executar el programa donat partint d'una memòria buida.
3. Afegiu una expressió `read` a SimpleLP que retorni el valor del següent enter de l'entrada.
4. Afegiu una instrucció del tipus `for i ← a .. b`.
5. Feu que `print` pugui escriure una llista de valors (`print a, b, a + b` per ex).
6. Escriviu l'AST d'aquest programa:

```
# factorial en SimpleLP
n ← read
f ← 1
for i ← 2 .. n
  f ← f * i
end
print n, f
```